# Minimum spanning trees and shortest paths

An MST minimizes total edge weight while connecting all vertices. A shortest path minimizes the cost between specified endpoints. Their states and safe choices differ.

Study **cuts and MSTs → relaxation, DAGs, Bellman–Ford, Dijkstra → APSP, min-plus, Floyd–Warshall, Johnson**.

Prerequisites are the disjoint-sets unit in this series (original chapter 21), priority-queue contracts, and chapter 7 graphs, paths, and topological order. Original chapter 21 covers MAKE-SET, UNION, FIND, rank, path compression, and the detailed inverse-Ackermann bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fix the problem before choosing an algorithm

| Problem and assumptions | Correctness argument |
| --- | --- |
| Minimum connection cost in an undirected graph | Cut property and safe-edge exchange. |
| Single-source paths in a DAG | Topological order completes predecessors first. |
| Single-source paths with negative edges | Bellman–Ford edge budgets and negative-cycle detection. |
| Single-source paths with nonnegative edges | Dijkstra's settled-vertex invariant. |
| All ordered endpoint pairs | DP states or Johnson's potential reweighting. |

Check the distance definition, graph assumptions, state trace, and cost model. Distinguish the effects of negative edges and negative cycles.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-01 | Minimize connection cost

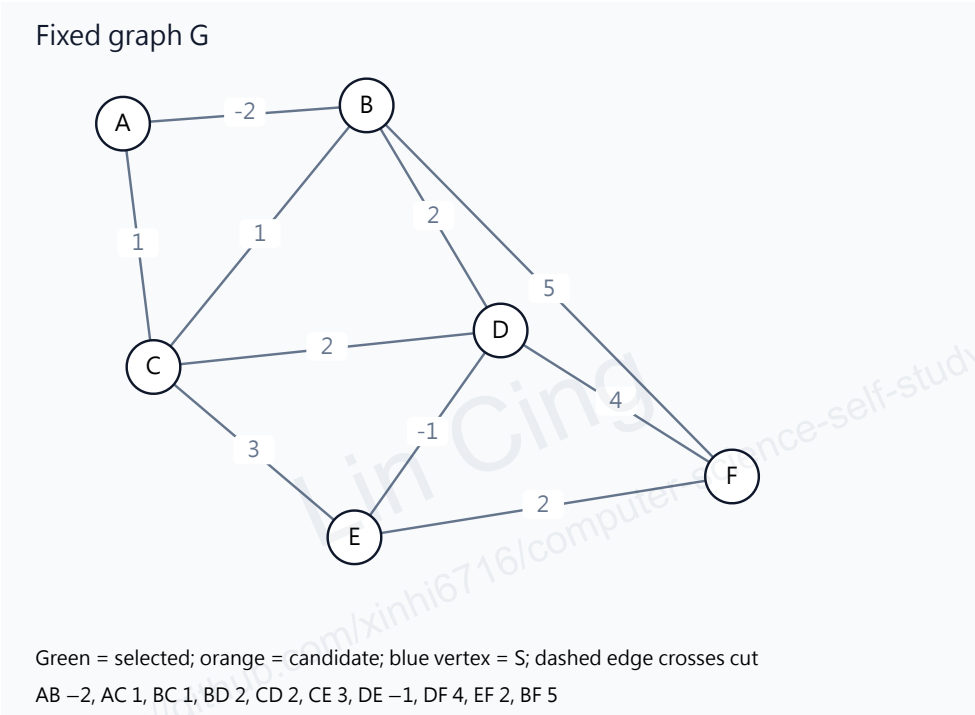
For an undirected weighted graph $G=(V,E)$, $w:E\to\mathbb{R}$, an MST is a spanning tree minimizing $\sum_{e\in T} w(e)$. A tree on all vertices has $|V|-1$ edges. MST paths need not be shortest paths from a root.

Prerequisites are MAKE-SET, UNION, FIND, rank and path compression from the disjoint-sets unit in this series (original chapter 21), heap contracts, and chapter 7 paths, cycles and cuts. The detailed DSU amortized analysis is in original chapter 21.

Sources [CLRS23 § 23.1; MIT04 L13.8–9; MIT05 L16.8–9].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-02 | One graph, two greedy rules




V={A,B,C,D,E,F}. AB=−2, AC=BC=1, BD=CD=EF=2, CE=3, DE=−1, DF=4, BF=5.

# A08-MST-03 | Safe preserves some optimum

Kruskal sorts by (weight,edge name). Prim starts at A and extracts by (key,vertex name). It changes a parent only for a strictly lower key. These conventions apply to every trace.

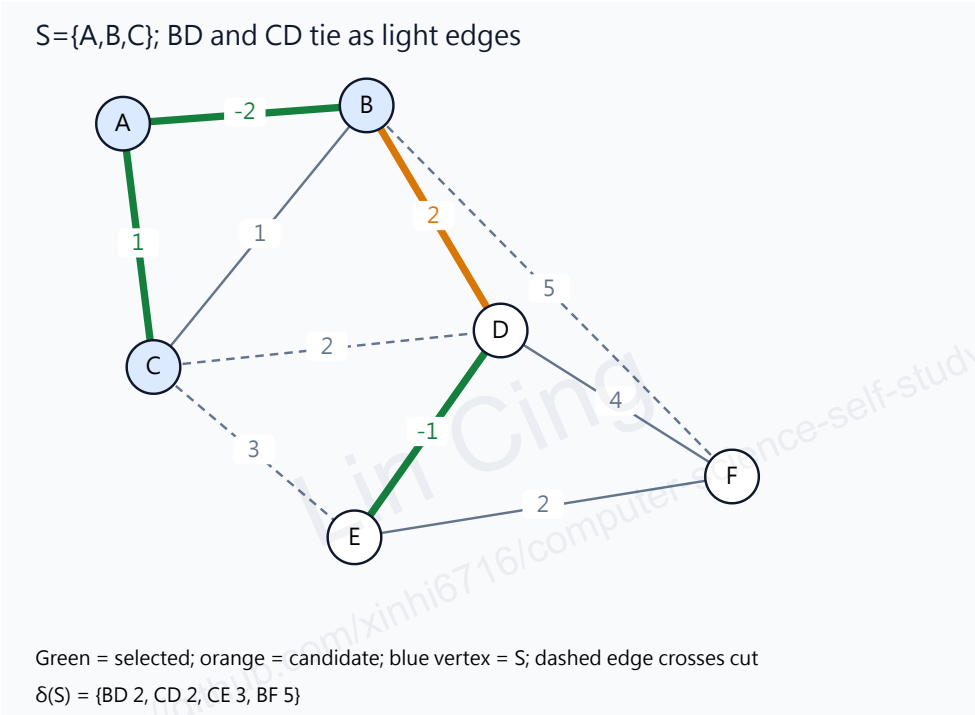
Maintain $A \subseteq$ some MST, rather than every MST. Edge e is safe for A when $A \cup \{e\}$ is contained in some MST.

A cut (S,V−S) respects A if no edge of A crosses it. A light edge has minimum weight among edges crossing that cut. Several light edges may tie.

Source [CLRS23 Theorem 23.1].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-04 | State the cut property with ties




If A extends to an MST and the cut respects A, any light crossing edge is safe for A. Here δ(ABC)={BD 2,CD 2,CE 3,BF 5}. Either BD or CD is safe.

# A08-MST-05 | Account for every exchange step

The unique light edge of a cut occurs in every MST. With ties, a light edge occurs in some MST; it need not occur in every MST.

MIT04/05 assume distinct weights for simplicity. We use the general CLRS statement.

Choose an MST $T$ containing $A$. If $e \in T$, we are done. Otherwise $T+e$ creates one cycle. Since $e$ crosses the cut, the $T$-path between its endpoints contains another crossing edge $f$.

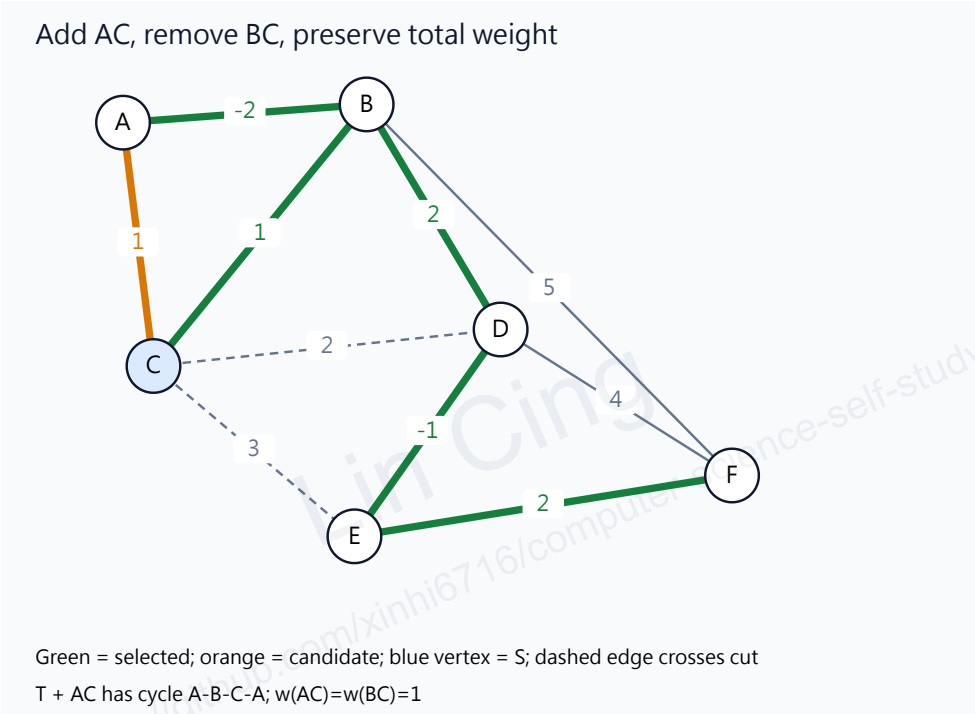
The cut respects $A$, so $f \notin A$. Since $e$ is light, $w(e) \leq w(f)$. $T' = T - f + e$ is a tree containing $A \cup \{e\}$, and $w(T') \leq w(T)$.

Optimality of $T$ forces equality. Ties are allowed. If $e$ is uniquely light, excluding $e$ would give a strictly cheaper tree and a contradiction.

Sources [CLRS23 pp627–628; MIT04 L13.22–25; MIT05 L16.22–25].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-06 | An exchange preserves A




Let A={AB,DE} and T={AB,BC,BD,DE,EF}. The cut S={C} respects A. AC and BC are both light, with weight 1. Adding AC creates A–B–C–A; removing BC preserves A.

# A08-MST-07 | The cycle property needs strictness

The result {AB,AC,BD,DE,EF} also weighs 2. Safe does not mean that the edge already belongs to the particular T we chose.

If e is the unique heaviest edge of a cycle, no MST contains e. If an MST T contained e, removing it would create a cut. Another cycle edge f crosses that cut and satisfies w(f)<w(e). Exchanging them would lower the weight.

If e only ties for heaviest, some MST excludes e. In a unit-weight triangle, every edge can occur in an MST and every edge can be omitted from one. Do not delete all tied edges at once.

Source [CLRS23 exercise 23.1-5, derived].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-08 | Kruskal tests current components

Scan edges in increasing (weight,edge name) order. Accept uv and UNION only when FIND(u)≠FIND(v). Otherwise an already selected path joins u and v, so accepting uv creates a cycle.

An accepted edge is light across the cut of a current component, which respects A. DSU represents connectivity of (V,A), not connectivity of the input graph.

Source [CLRS23 § 23.2].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-09 | All ten Kruskal rounds

| Round | Edge | Decision | DSU components after round |
|---|---|---|---|
| 0 | ∅ | initialize | A / B / C / D / E / F |
| 1 | AB, −2 | accept | AB / C / D / E / F |
| 2 | DE, −1 | accept | AB / C / DE / F |
| 3 | AC, 1 | accept | ABC / DE / F |
| 4 | BC, 1 | reject, B–A–C already connected | ABC / DE / F |
| 5 | BD, 2 | accept | ABCDE / F |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

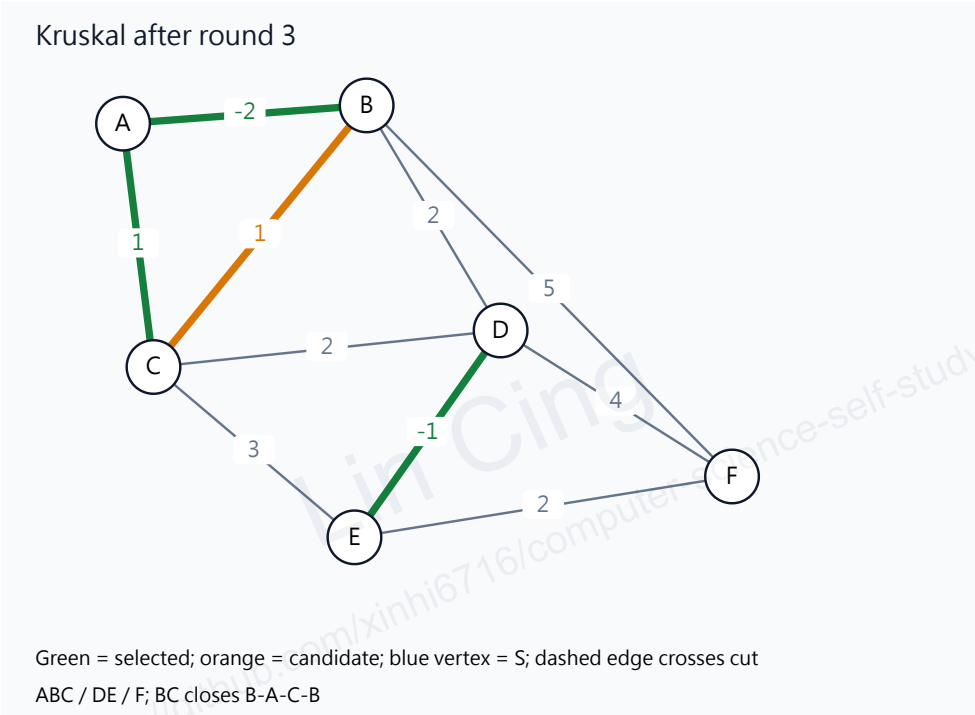
# A08-MST-09B | Kruskal: rounds 6–10

| Round | Edge | Decision | DSU components after round |
|---|---|---|---|
| 6 | CD, 2 | reject, C–A–B–D already connected | ABCDE / F |
| 7 | EF, 2 | accept | ABCDEF |
| 8 | CE, 3 | reject, C–A–B–D–E already connected | ABCDEF |
| 9 | DF, 4 | reject, D–E–F already connected | ABCDEF |
| 10 | BF, 5 | reject, B–D–E–F already connected | ABCDEF |

Round 7 reaches five accepted edges, so an implementation may stop there. This table scans all ten edges to expose every rejection. Components are semantic sets, not the internal DSU parent trees.

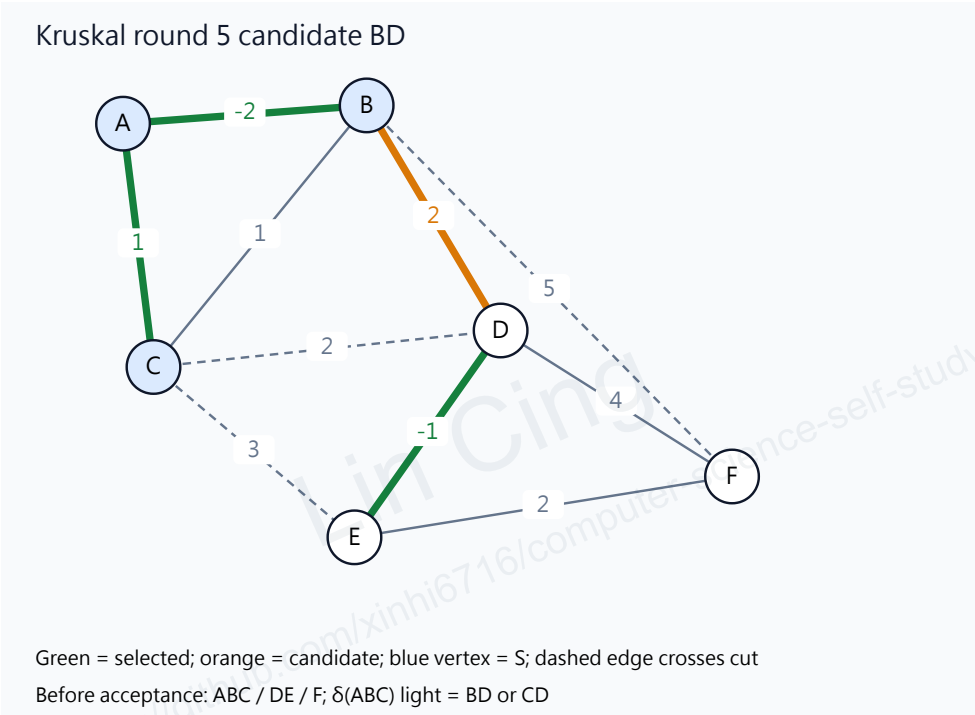
Lin Cing https://github.com/xinhi6716/computer-science-self-study

# A08-MST-10 | Reject BC, accept BD




Round 4 rejects BC because its endpoints share a component, not because it is expensive.

# A08-MST-10 | Reject BC, accept BD (continued)




The round 5 cut is the cut from card 04. CD has equal weight and could replace BD under a different tie order. It is considered later and then closes a cycle.

# A08-MST-11 | A Prim key prices one edge

S contains extracted vertices. For $v \notin S$, key[v]=min{w(u,v):$u \in S$}, or $\infty$ if no such edge exists. parent[v] records a vertex attaining this minimum.

The minimum-key vertex contributes a light edge across (S,V−S). Update v only while $v \in Q$ and w(u,v) <key[v]. Initial key[A]=0 chooses the root and is not an edge.

Sources [CLRS23 pp634–636; MIT04 L13.26; MIT05 L16.26].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

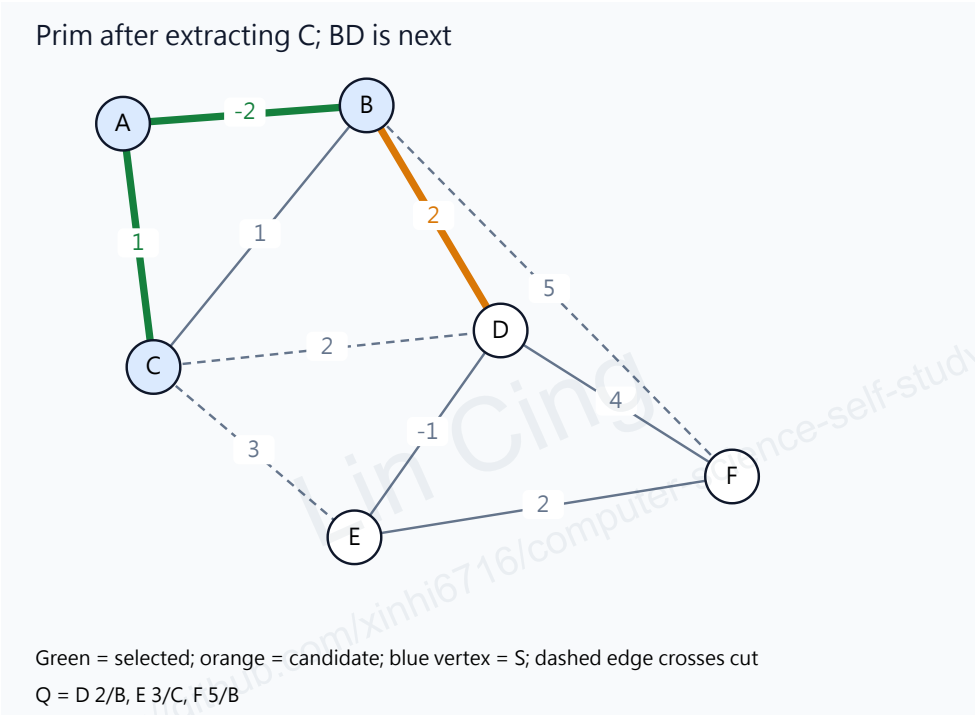
# A08-MST-12 | Every Prim key, parent and queue

| Extract | S before extraction | Add | Q after relaxation, key/parent, in priority order |
|---|---|---|---|
| Initial | ∅ | ∅ | A 0/∅, B ∞/∅, C ∞/∅, D ∞/∅, E ∞/∅, F ∞/∅ |
| A | ∅ | root | B −2/A, C 1/A, D ∞/∅, E ∞/∅, F ∞/∅ |
| B | A | AB | C 1/A, D 2/B, F 5/B, E ∞/∅ |
| C | AB | AC | D 2/B, E 3/C, F 5/B |
| D | ABC | BD | E −1/D, F 4/D |
| E | ABCD | DE | F 2/E |
| F | ABCDE | EF | ∅ |

Q is written in logical priority order, not heap-array order. Vertices absent from Q belong to S. Final parents are B←A,C←A,D←B,E←D,F←E.

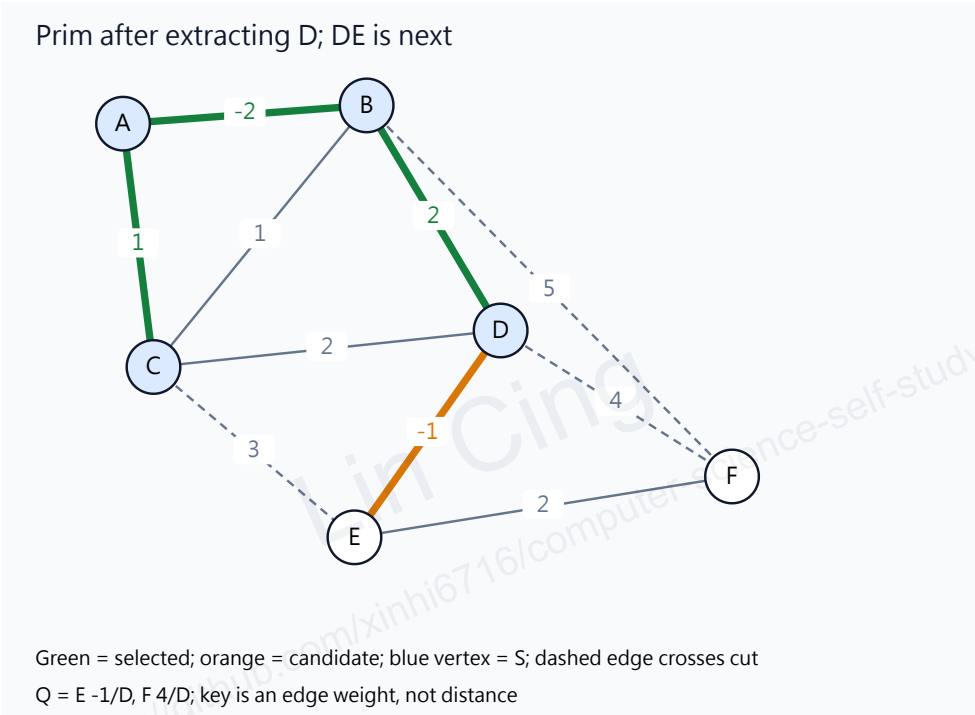
Lin Cing https://github.com/xinhi6716/computer-science-self-study

# A08-MST-13 | Each extraction has a cut




After extracting C, S=ABC and Q={D,E,F}. Its crossing edges are BD 2,CD 2,CE 3,BF 5. D has key 2 and parent B. Since CD=2 is not a strict improvement, B stays its parent.

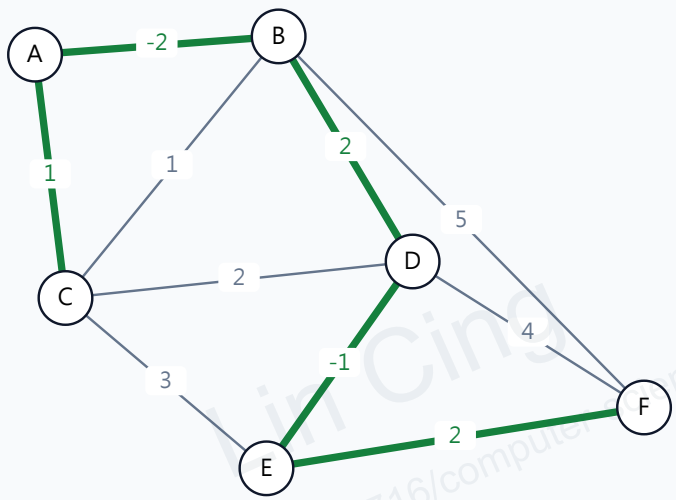
# A08-MST-14 | Negative weights are valid




D is extracted at key 2. Scanning D then decreases E to −1. The extracted key sequence 0,−2,1,2,−1,2 is not monotone.

# A08-MST-15 | Shared result, four optima

One MST, total weight 2




Green = selected; orange = candidate; blue vertex = S; dashed edge crosses cut

Other MSTs replace AC by BC and/or BD by CD

T={AB,AC,BD,DE,EF}, with weight −2+1+2−1+2=2. Independent enumeration of all C(10,5)=252 five-edge subsets, followed by the tree test, finds minimum weight 2 and exactly four MSTs.

# A08-MST-15A | Cuts, negative weights and optima

The other cuts before extraction have S={A},AB,ABCD,ABCDE. Their light candidates are AB −2, AC/BC 1, DE −1, EF 2 respectively. Root A contributes no parent edge.

The exchange proof compares edge weights and never requires nonnegative weights. Prim does not relax key[u]+w(u,v), unlike a shortest-path distance update.

All optima are {AB,DE,EF}∪{AC or BC}∪{BD or CD}. Distinct weights suffice for uniqueness. Repeated weights do not by themselves prove non-uniqueness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-16 | Disconnected input yields a forest

If G has c components, no spanning tree covers all vertices. An MSF combines an MST of each component and has $|V|-c$ edges.

Keep only {AB −2,AC 1,BC 1,DE −1,DF 4,EF 2} from our graph. Components are ABC and DEF. One MSF is {AB,AC,DE,EF}, weight 0. Replacing AC with BC gives another.

Kruskal yields an MSF by scanning all edges. In Prim, an extracted minimum key of $\infty$ starts a new root with parent=$\varnothing$. Add no artificial infinite-weight edge, then scan its neighbors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A08-MST-17 | Name the implementation with the bound

Write n=|V| and m=|E|. Kruskal uses O(m log m) comparison sorting and O((n+m)(1+α(n))) DSU time from original chapter 21. For a simple connected graph, O(m log n) is the combined bound.

| Prim implementation | Extract minimum | Decrease key | Total time |
|---|---|---|---|
| Array scan + adjacency matrix | O(n) | O(1) | $O(n^2)$ |
| Indexed binary heap + adjacency lists | O(log n) | O(log n) | O((n+m) log n) |
| Fibonacci heap + adjacency lists | O(log n) amortized | O(1) amortized | O(m+n log n) |

An indexed heap maintains vertex positions and O(1) Q membership. A lazy heap pushes a new record on each improvement and discards stale records at pop. It can hold O(m) records, giving O(n+m log(m+1)) time and O(n+m) auxiliary space; its heap need not have only n entries.

Sources [CLRS23 pp633,636; MIT04 L13.40–50; MIT05 L16.40–50].

# A08-MST-18 | Cross-check proof, state and result

For acceptance, identify a cut respecting A and a light edge. For rejection, exhibit an already selected path. Read Prim key/parent/Q after every round and Kruskal components after every round. Ties choose between optima; negative weights do not invalidate safety.

Exercises A08-M01–08 check cuts, exchanges, cycles, DSU, PQ, MSF, uniqueness and implementation costs. `check.py` independently enumerates spanning trees and verifies both complete traces.

Original handwritten PDF pp36–37 motivates exchanges and a counterexample for the third-lightest edge. CLRS23 supplies the precise respecting-cut and tie conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Three meanings of a distance

For a finite directed graph with finite real edge weights, define $\delta(s,v)$ as the infimum of costs of all s-to-v walks. It is $\infty$ when v is unreachable, $-\infty$ when a reachable negative cycle can be visited on the way to v, and otherwise a finite minimum. The empty walk gives $\delta(s,s) \leq 0$.

We track d, $\pi$, and the queue on fixed graphs to identify when an algorithm can promise a final answer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Negative edges versus negative cycles

If v is reachable and no s-to-v walk can visit a negative cycle, delete positive cycles to improve its cost and zero cycles to preserve it. A simple shortest path with at most $|V|-1$ edges therefore exists.

For reachable v, the distance is finite exactly when there is no negative cycle reachable from s that can reach v. A negative cycle on another branch need not affect v. For example, $s \to a=1$, $a \to a=1$, $a \to v=1$ gives $\delta(s,v)=2$ despite the positive cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The shared relaxation operation

```
initialize: d[s]=0; d[v≠s]=∞; π[v]=NIL
RELAX(u,v,w):
  if d[u] ≠ ∞ and d[v] > d[u]+w:
    d[v]=d[u]+w; π[v]=u
```

Represent ∞ separately. For finite w, ∞+w=∞. Check reachability before adding fixed-width numbers and prevent overflow. Never add to a maximum-integer sentinel or clamp negative values to an arbitrary floor. Update π only on strict improvement; store edge IDs for parallel arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Relaxation invariants and convergence

Every finite $d[v]$ is the cost of an actual s-to-v walk, so $d[v] \geq \delta(s,v)$. Labels never increase and unreachable vertices stay at $\infty$.

If $s \rightsquigarrow u \to v$ is a shortest path and its prefix has converged, relaxing $(u,v)$ makes v converge too. Relax the edges of a shortest path in path order to propagate equality, even with other relaxations interleaved. Once a finite optimum is attained, the label stays there.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fixed graph G+ and initialization

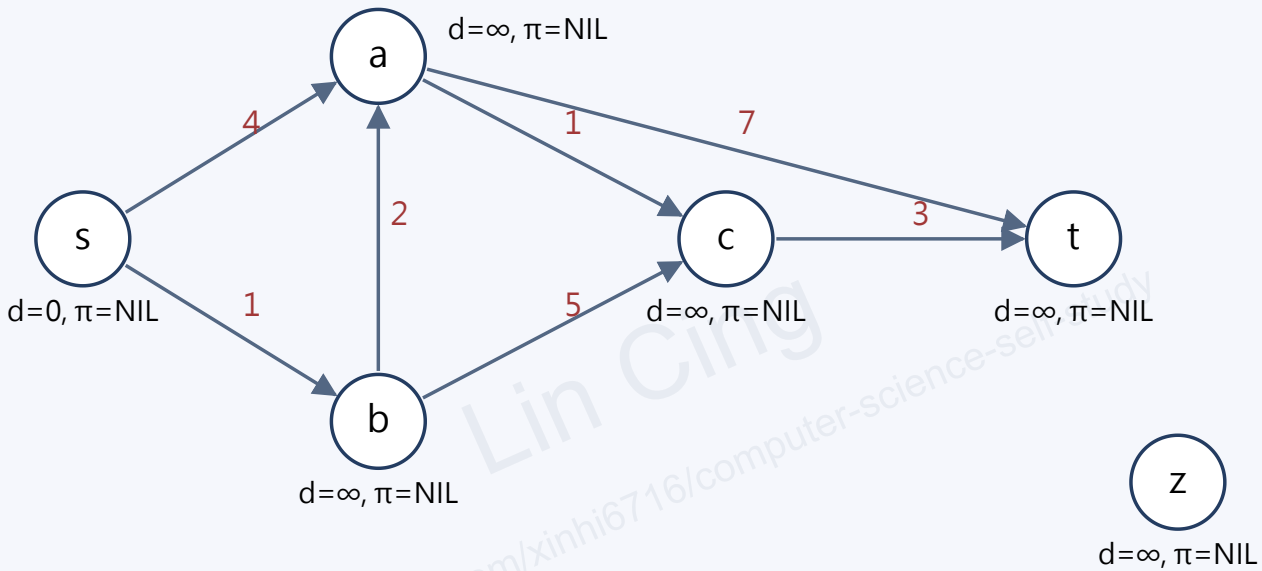
Use vertex order (s,a,b,c,t,z); z is isolated. The complete edge list is

s→a:4, s→b:1, b→a:2, a→c:1, b→c:5, c→t:3, a→t:7.

Initially d=(0,∞,∞,∞,∞,∞) and every π is NIL. These graphs and numerical traces are original teaching examples.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fixed graph G+ and initialization (continued)




Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A DAG can have negative edges

Graph $G-$ changes only $a \to c$ to $-2$. It remains a DAG. Fix topological order s,b,a,c,t,z. Before processing u, all predecessors are finished, so d[u] is final; no minimum-key choice is required.

Topological sorting and scanning all arcs take $\Theta(V+E)$. Validate acyclicity first. An arbitrary ordering of a cyclic graph cannot serve as a topological order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The DAG’ s single propagation pass

Each row follows relaxation of all outgoing arcs. Both vectors use (s,a,b,c,t,z).

| u | d | π |
|---|---|---|
| initial | (0,∞,∞,∞,∞,∞) | (NIL,NIL,NIL,NIL,NIL,NIL) |
| s | (0,4,1,∞,∞,∞) | (NIL,s,s,NIL,NIL,NIL) |
| b | (0,3,1,6,∞,∞) | (NIL,b,s,b,NIL,NIL) |
| a | (0,3,1,1,10,∞) | (NIL,b,s,a,a,NIL) |
| c | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |
| t,z | (0,3,1,1,4,∞) | (NIL,b,s,a,c,NIL) |

Although a has distance 3 and c ultimately has 1, a must be processed first.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Bellman-Ford with a fixed edge order

Run on $G^-$ without using its topological order. Each pass scans exactly these seven arcs and writes labels in place.

$c \to t:3$, $a \to t:7$, $a \to c:-2$, $b \to c:5$, $b \to a:2$, $s \to b:1$, $s \to a:4$.

Run $|V|-1=5$ passes, then check for any relaxable arc. A whole pass with no changes allows early stopping. Edge order changes intermediate states and convergence speed, but not the final answer when no reachable negative cycle exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Bellman-Ford’s complete pass trace

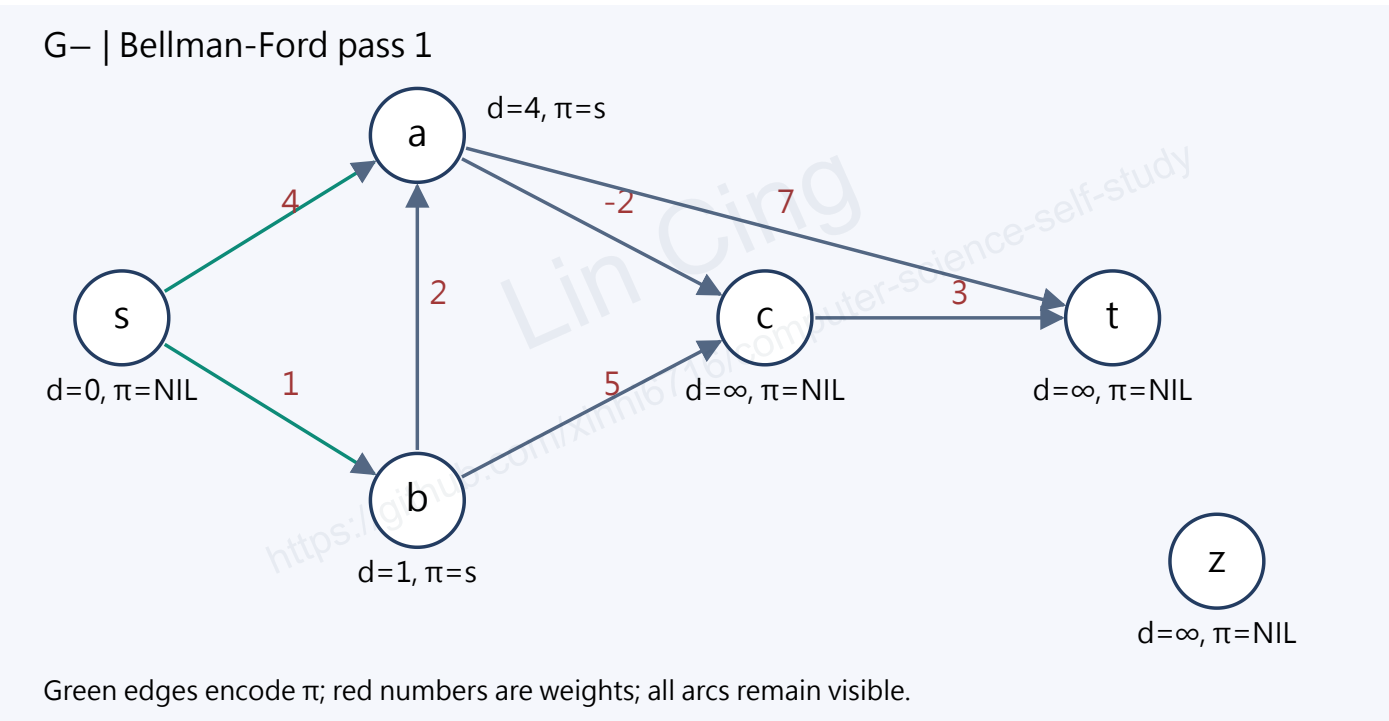
| pass | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) |
|---|---|---|
| 0 | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) |
| 1 | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) |
| 2 | (0, 3, 1, 2, 11, ∞) | (NIL, b, s, a, a, NIL) |
| 3 | (0, 3, 1, 1, 5, ∞) | (NIL, b, s, a, c, NIL) |
| 4 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |
| 5 | (0, 3, 1, 1, 4, ∞) | (NIL, b, s, a, c, NIL) |

Pass 1 reaches s’s outgoing arcs only at its end. Pass 2 first uses a=4 to obtain c=2,t=11, then lowers a to 3. Pass 3 first lowers t to 5 and only later lowers c to 1. Pass 4 finally lowers t to 4.

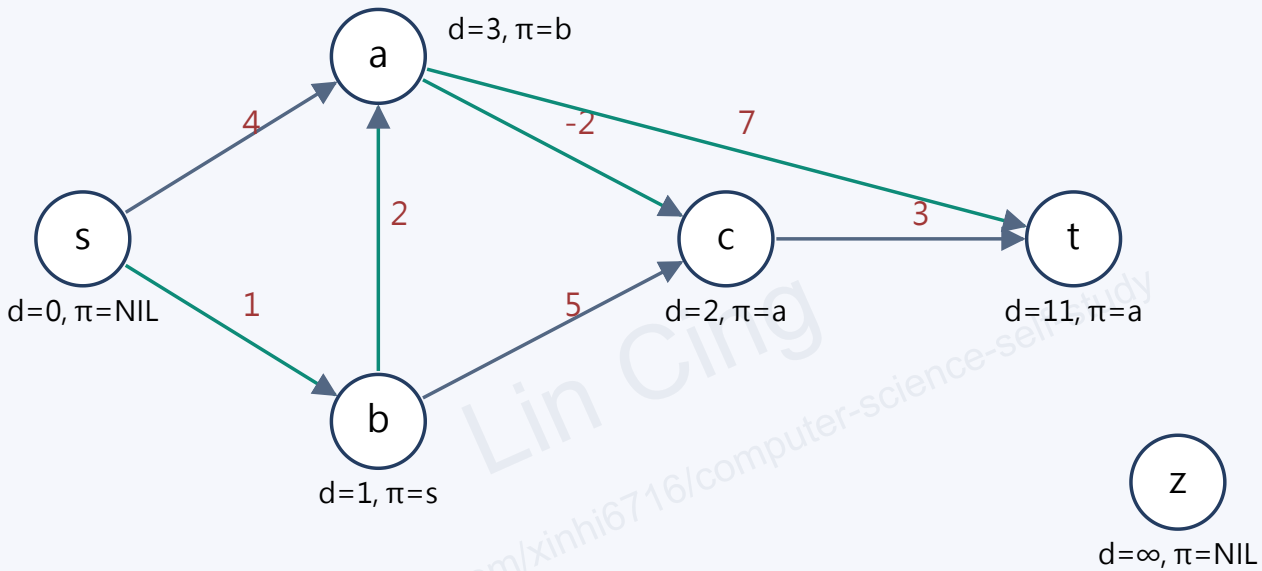
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Pass diagrams retain transient labels

These figures use the same G−, edge order, and sampling points as the table. After pass 2, π already describes s→b→a→c but d[c]=2 differs from that current parent-chain cost, 1. Parent chains and labels can be temporarily out of sync.



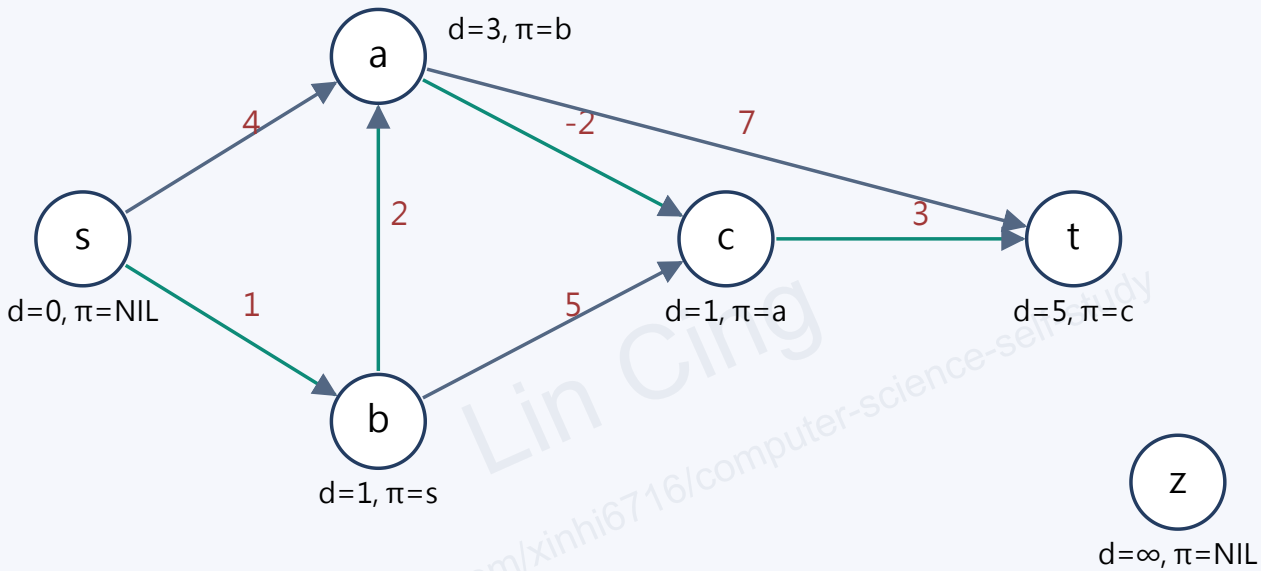
# Pass diagrams retain transient labels (continued)




Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Pass diagrams retain transient labels (continued)




Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Why V−1 passes suffice

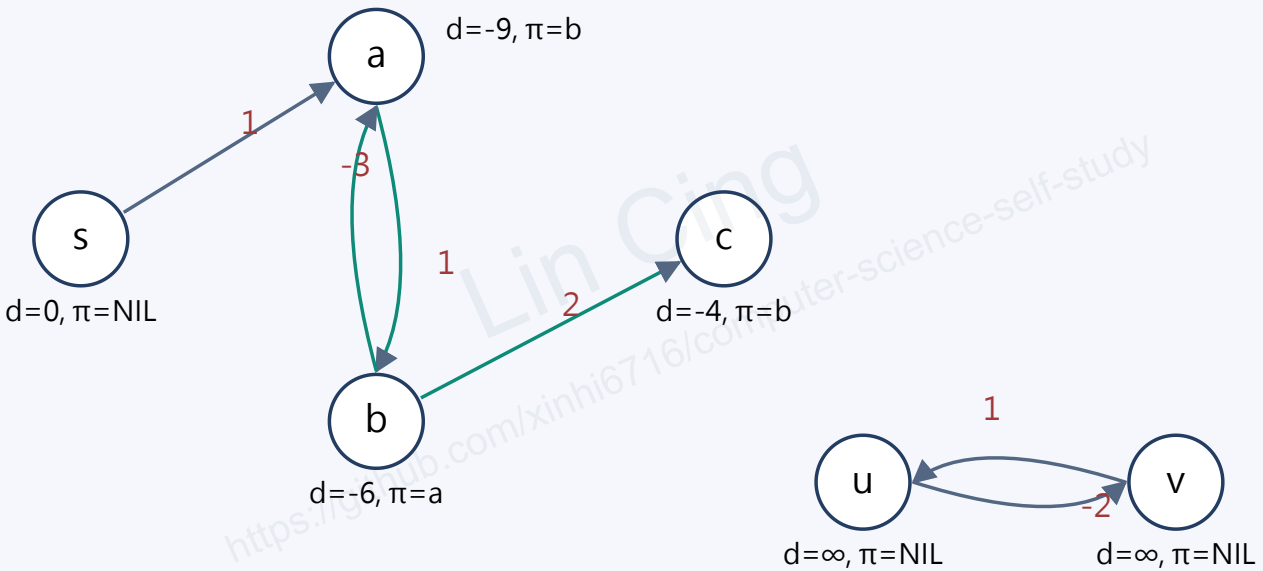
After complete pass k, every vertex with a shortest path of at most k arcs has converged. Pass k scans that path’ s kth arc, and its prefix converged by the previous pass.

An in-place pass can propagate across several arcs, so d is not necessarily the exact optimum restricted to k arcs. Without reachable negative cycles, a simple shortest path uses at most V−1 arcs. The time is O(VE); including initialization, O(V+VE) also covers E=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Reachable negative cycles and descendants

Graph N has exactly s→a:1,a→b:1,b→a:−3,b→c:2,u→v:−2,v→u:1. Vertices u,v are unreachable. Fix that edge order and vertex order (s,a,b,c,u,v).




Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Detect the negative cycle and mark affected vertices

Passes 0 through 5 give d=(0,∞,∞,∞,∞,∞), (0,−1,2,4,∞,∞), (0,−3,0,2,∞,∞), (0,−5,−2,0,∞,∞), (0,−7,−4,−2,∞,∞), (0,−9,−6,−4,∞,∞).

A read-only sixth check finds a→b relaxable because −9+1<−6. Start DFS/BFS from every relaxable arc’s destination and mark its descendants −∞. The affected set is {a,b,c}; s remains 0 and u,v stay ∞. Detection is justified by summing the supposedly nonrelaxable inequalities around a reachable negative cycle, contradicting its negative weight.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Dijkstra’ s contract

Every edge weight must be nonnegative. S contains settled vertices and Q=V−S. Extract a minimum-key u, add it to S, relax all outgoing arcs, and perform decrease-key when needed.

Our queue contains every vertex and breaks equal-key ties by vertex name. Tables display sorted logical queue contents, not heap array positions. Stop early if the minimum is ∞; all remaining vertices are unreachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Dijkstra’s full d, π, and queue trace

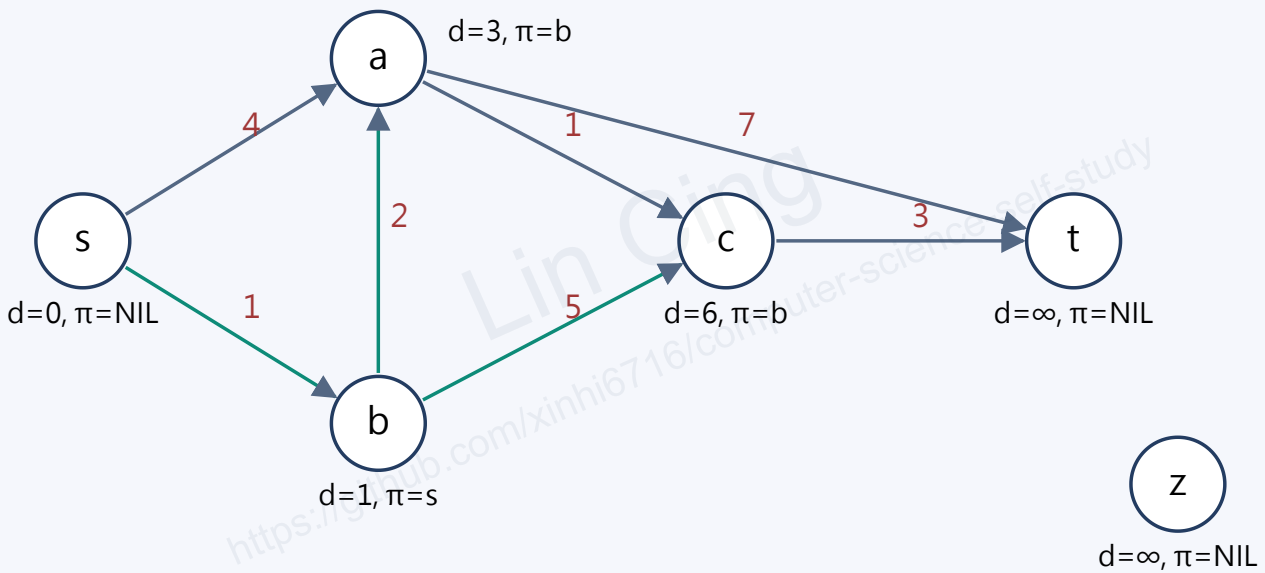
| extract | d(s,a,b,c,t,z) | π(s,a,b,c,t,z) | Q after relaxation |
|---|---|---|---|
| init | (0, ∞, ∞, ∞, ∞, ∞) | (NIL, NIL, NIL, NIL, NIL, NIL) | s:0, a:∞, b:∞, c:∞, t:∞, z:∞ |
| s | (0, 4, 1, ∞, ∞, ∞) | (NIL, s, s, NIL, NIL, NIL) | b:1, a:4, c:∞, t:∞, z:∞ |
| b | (0, 3, 1, 6, ∞, ∞) | (NIL, b, s, b, NIL, NIL) | a:3, c:6, t:∞, z:∞ |
| a | (0, 3, 1, 4, 10, ∞) | (NIL, b, s, a, a, NIL) | c:4, t:10, z:∞ |
| c | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | t:7, z:∞ |
| t | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | z:∞ |
| z | (0, 3, 1, 4, 7, ∞) | (NIL, b, s, a, c, NIL) | |

Each row samples the state after relaxing all outgoing arcs of the extracted vertex. Extracting s gives a=4,b=1; extracting b lowers a to 3 and first reaches c at 6.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

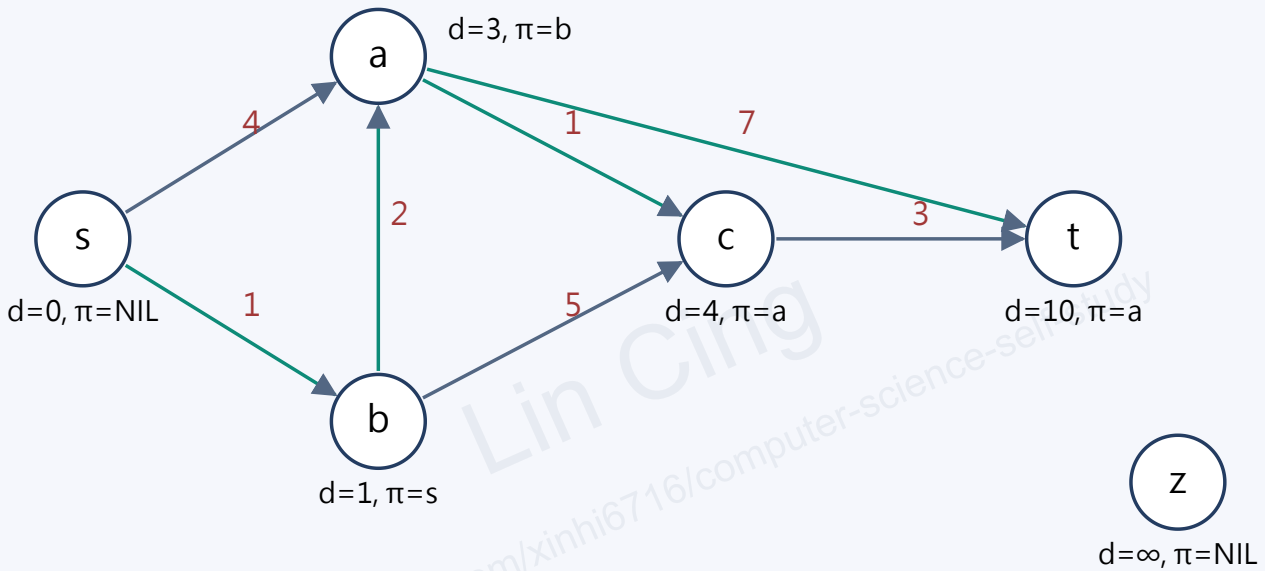
# Two visible queue updates

After extracting b, a=3 and c=6. After extracting a, c=4 and t=10. Thus π[c] changes from b to a and t first obtains parent a.




Green edges encode π; red numbers are weights; all arcs remain visible.

# Two visible queue updates (continued)

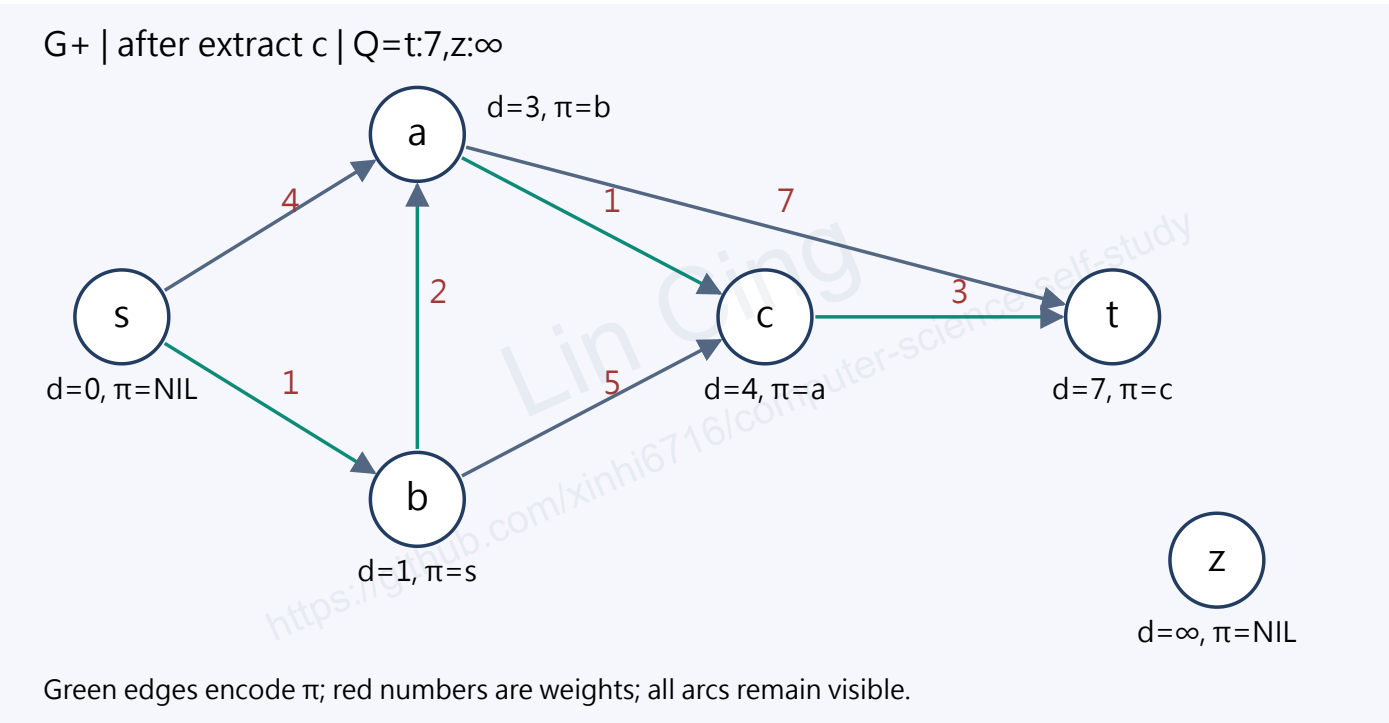



Green edges encode π; red numbers are weights; all arcs remain visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The last improvement and unreachability

Extracting c lowers t from 10 to 7 and changes π[t] from a to c. Extracting t changes nothing. The remaining ∞ at z is the correct answer.




The shortest path s→b→a→c→t costs 1+2+1+3=7.

# Why extract-min settles a vertex

Invariant: $d[x]=\delta(s,x)$ for every $x \in S$. For the next reachable u, take a shortest s-to-u path. Let y be its first vertex outside S and x its predecessor. Relaxing x’ s outgoing arcs already established $d[y]=\delta(s,y)$.

The nonnegative y-to-u suffix gives $\delta(s,y) \leq \delta(s,u)$. Minimum extraction and the upper-bound property give $\delta(s,u) \leq d[u] \leq d[y]=\delta(s,y) \leq \delta(s,u)$. Every inequality is equality. For unreachable u, use the no-path property.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# How a negative edge breaks settlement

Counterexample H has $s \to a:2, s \to b:5, b \to a:-4, a \to t:2$. Extraction order is s,a,t,b. Vertex a is settled at 2 although $\delta(s,a)=1$; t is settled at 4 although its optimum is 3.

Ignoring updates to settled vertices leaves a wrong. Allowing a later update to a without rescanning its outgoing arcs still leaves t wrong. Reopening vertices requires a different proof and cost bound. The failed proof inequality is $\delta(s,b) \leq \delta(s,a)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Priority queue costs

There are |V| extract-min operations, at most |E| successful decrease-key operations, and one scan of each arc.

| Queue | Total time |
|---|---|
| Unsorted array | $O(V^2+E)$ |
| Binary heap | $O((V+E) \log V)$ |
| Fibonacci heap | $O(E+V \log V)$ amortized |

A lazy heap inserts a record on each improvement and skips stale extractions. Its bound is $O((V+E) \log(V+E))$ time and $O(V+E)$ space. Do not attribute the $O(V)$ heap space of decrease-key to this variant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Path reconstruction and algorithm choice

Reconstruct only confirmed finite distances. Follow $\pi$ backward from the target to s, then reverse. Return unreachable for $\infty$ and no finite shortest path for $-\infty$. A $|V|$-step guard reveals corrupted or unfinished parent chains.

In G−, t←c←a←b←s reverses to s→b→a→c→t of cost 1+2−2+3=4. Source s returns the empty path.

| Graph condition | Algorithm |
| --- | --- |
| DAG, negative arcs allowed | Topological relaxation |
| General graph, all arcs nonnegative | Dijkstra |
| General graph, negative arcs possible | Bellman-Ford and reachable-cycle check |

Sources are original notes p42, CLRS Chapter 24, MIT04 L14/L15, and MIT05 L17/L18. See sources.json for PDF and printed-page mappings.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# APSP returns a distance and path for each ordered pair

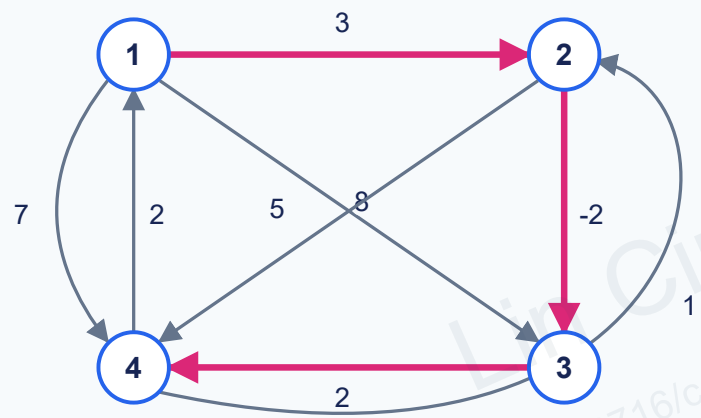
Input is a directed graph $G=(V,E)$, with $n=|V|$ and $m=|E|$. Negative edges are allowed. Each row of the distance matrix fixes a source. Unreachable pairs have distance $+\infty$. A pair that can traverse a negative cycle has distance $-\infty$ and no finite shortest path.

First assume no negative cycle. $W$ stores the minimum direct-edge weight, $\infty$ for absent edges, and $\min(0,\text{self-loop weight})$ on the diagonal. Never erase a negative self-loop.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# APSP returns a distance and path for each ordered pair (continued)

**Shared four-vertex graph G**




8 edges; negative edge 2→3

2→3→4→2 = 1

1→2→3→4 = 3

1→4 = 7

3 < 7

Pink edges mark the path or negative cycle. Arrows show direction.

This four-vertex example is original, not a textbook figure. Sources [S1 § 25.1–25.3], [S2 p4–23], and [S3 p4–23].

# The edge-budget state means at most t edges

$L^{(t)}[i,j]$ is the minimum weight of a walk with at most t edges. $L^{(0)}=I$, whose diagonal is 0 and other entries are $\infty$.

$$L^{(t+1)}[i,j] = \min_k \{L^{(t)}[i,k] + W[k,j]\}.$$

Split a nonempty walk before its final edge; each candidate also concatenates into a feasible walk. The diagonal W[j,j] is the minimum of 0 and any self-loop weight, so $W[j,j] \leq 0$. For a walk using at most t edges, append its chosen diagonal: the empty walk if it is 0, or a negative self-loop otherwise. The result uses at most t+1 edges and costs no more. Thus shorter walks are covered even with negative cycles; a fixed budget need not give unrestricted distances.

Without negative cycles, deleting a nonnegative cycle cannot raise weight. A shortest path can be simple, using at most n−1 edges. This justifies the stopping budget. [S1 p687–688; S2 p6–9; S3 p6–9]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Min-plus multiplication adds edge budgets

Define $(A \otimes B)[i,j] = \min_k(A[i,k]+B[k,j])$. For $A=L^{(a)}$ and $B=L^{(b)}$, $A \otimes B = L^{(a+b)}$. Split any walk with at most $a+b$ edges after $\min(a,\text{its length})$ edges. Conversely, concatenate the two feasible pieces.

Associativity follows by minimizing the same expression.

$$(A \otimes B) \otimes C = A \otimes (B \otimes C), \qquad \min_{p,q}(A[i,p] + B[p,q] + C[q,j]).$$

I is the identity. Use finite numbers and $+\infty$ here, so $\infty+(-\infty)$ never arises. Replace sum and product in ordinary matrix multiplication by minimum and addition. Strassen subtraction does not transfer directly. [S1 p688–691; S2 p10–14; S3 p10–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Slow and faster change the number of stages

```
Extend(A,B):
  C = new n×n matrix filled with ∞
  for i,j,k: C[i,j] = min(C[i,j], A[i,k]+B[k,j])
  return C
Slow(W):
  L = I
  repeat n−1 times: L = Extend(L,W)
  return L
Faster(W):
  L = W; t = 1
  while t < n−1: L = Extend(L,L); t = 2*t
  return L
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Min-plus costs and negative-cycle detection

One Extend takes $\Theta(n^3)$. Slow performs $n-1$ products, taking $\Theta(n^4)$. Faster uses budgets 1,2,4,… and takes $\Theta(n^3 \log n)$. Retaining only the inputs and a fresh output uses $\Theta(n^2)$ working space. Do not overwrite an input while computing its product.

These answers assume no negative cycle. To detect one from a min-plus diagonal, use a budget of at least n. A negative simple cycle may contain n edges. The $n-1$ stopping rule applies to distances when no negative cycle exists. [S1 p689–692; S2 p13–14; S3 p13–14]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Compute $L^{(2)}$ and $L^{(3)}$ on the same graph

$L^{(1)}=W$. $L^{(2)}$ tries every possible final intermediate vertex.

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | 7 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(3)}$ already contains all distances because $n-1=3$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Compute $L^{(2)}$ and $L^{(3)}$ on the same graph (continued)

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

$L^{(2)}[1,4]=\min(7,3+5,8+2)=7$. $L^{(3)}[1,4]=1+2=3$ along $1\to2\to3\to4$. Faster computes $L^{(2)}$ and $L^{(4)}$; the latter equals $L^{(3)}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Floyd–Warshall limits the set of intermediate vertices

Every intermediate vertex of $D^{(k)}[i,j]$ belongs to $S_k=\{1,\ldots,k\}$. Endpoints i,j need not belong to $S_k$. $D^{(0)}=W$.

$$D^{(k)}[i,j] = \min\{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}.$$

Without negative cycles, choose a simple shortest path. If it avoids k, use the first term. Otherwise split at its unique occurrence of k. Both subpaths have intermediates in $S_{k-1}$. Conversely, every candidate concatenates into an allowed walk, and removing nonnegative cycles remains feasible. Both directions prove equality.

The original notes motivate optimal substructure using shortest subpaths. The allowed set makes that observation into an executable DP. [S0 p18–19; S1 p693–695; S2 p15–17; S3 p15–17]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=0 allows no intermediate vertex

$D^{(0)}$ contains only direct edges and empty paths. Π[i,j] stores the vertex immediately before j, not a split vertex. It is NIL for i=j or no path; a direct edge has predecessor i.

| $D^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | ∞ | 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=0 allows no intermediate vertex (continued)

| $\Pi^{(0)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | NIL | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=1 allows intermediate vertex 1

Only 4→3 improves. Its old value ∞ becomes 2+8=10 along 4→1→3. $\Pi^{(1)}[4,3]=\Pi^{(0)}[1,3]=1$. Endpoint 3 need not belong to {1}.

| $D^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 8 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | 10 | 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=1 allows intermediate vertex 1 (continued)

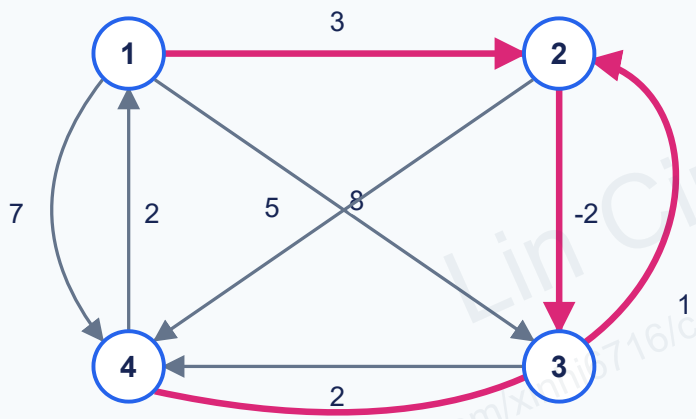
| $\Pi^{(1)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 1 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 1 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=2 improves two pairs through the negative edge

1→3 falls from 8 to 1; 4→3 falls from 10 to −1. Both end with 2→3, so both predecessors become 2.

**Improvements through intermediate 2**




$S_2 = \{1, 2\}$

$1 \to 3$: $8 \to 3 + (-2) = 1$

$4 \to 3$: $10 \to 1 + (-2) = -1$

Endpoint 3 need not be in $S_2$

$1 \to 4$ remains 7

Pink edges mark the path or negative cycle. Arrows show direction.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# k=2 improves two pairs through the negative edge (continued)

| $D^{(2)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 7 |
| 2 | ∞ | 0 | -2 | 5 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

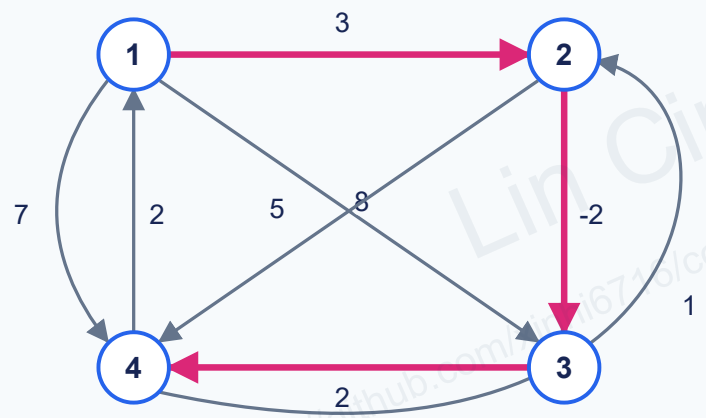
| $\Pi^{(2)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 1 |
| 2 | NIL | NIL | 2 | 2 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=3 first exposes the three-edge path

1→4 falls from 7 to 3; 2→4 falls from 5 to 0. Π[1,4]=Π[3,4]=3. Copying k itself is not a valid general predecessor rule.

### Improvements through intermediate 3




$S_3 = \{1, 2, 3\}$

1→4: 7 → 1 + 2 = 3

2→4: 5 → −2 + 2 = 0

4→2→3→4 = 1

Nonnegative cycle keeps diagonal 0

Pink edges mark the path or negative cycle. Arrows show direction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=3 first exposes the three-edge path (continued)

| $D^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | ∞ | 0 | -2 | 0 |
| 3 | ∞ | ∞ | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

| $\Pi^{(3)}$ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | NIL | NIL | 2 | 3 |
| 3 | NIL | NIL | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=4 finishes distances and reconstructs a path

New entries are 2→1=2, 3→1=4, and 3→2=3. From source 1, trace backward from 4 using Π[1,4]=3, Π[1,3]=2, and Π[1,2]=1. Reverse to obtain 1→2→3→4 of weight 3.

On strict improvement set Π[i,j]=Π[k,j]; retain the old predecessor on a tie. Check reachability and negative-cycle influence before reconstruction. Updating on ties can erase a valid predecessor through a zero-weight cycle. [S1 p695–697]

| D⁽4⁾ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 1 | 3 |
| 2 | 2 | 0 | -2 | 0 |
| 3 | 4 | 3 | 0 | 2 |
| 4 | 2 | 1 | -1 | 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# k=4 finishes distances and reconstructs a path (continued)

| Π⁽⁴⁾ i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | NIL | 1 | 2 | 3 |
| 2 | 4 | NIL | 2 | 3 |
| 3 | 4 | 4 | NIL | 3 |
| 4 | 4 | 4 | 2 | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Keep k outermost to preserve the DP stage

```
D = W
for k = 1..n:
  for i = 1..n:
    for j = 1..n:
      if D[i,k]+D[k,j] < D[i,j]:
        D[i,j] = D[i,k]+D[k,j]
        Π[i,j] = Π[k,j]
```

After outer stage k, every pair has incorporated $S_k$. Without negative cycles, D[k,k]=0. Row k and column k do not change during this stage, so in-place operands retain the required previous-stage values. Swap i and j freely. Moving k inside has no such general guarantee.

Counterexample has only 1→4=1, 4→3=1, and 3→2=1. One increasing i,j,k pass examines (1,2) before D[1,3] becomes finite, leaving ∞. Later it finds D[1,3]=2 but never revisits (1,2). The true distance is 3. [S1 p695,699 ex25.2-4; S2 p17; S3 p17]

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# A negative diagonal warns; mark −∞ by pair

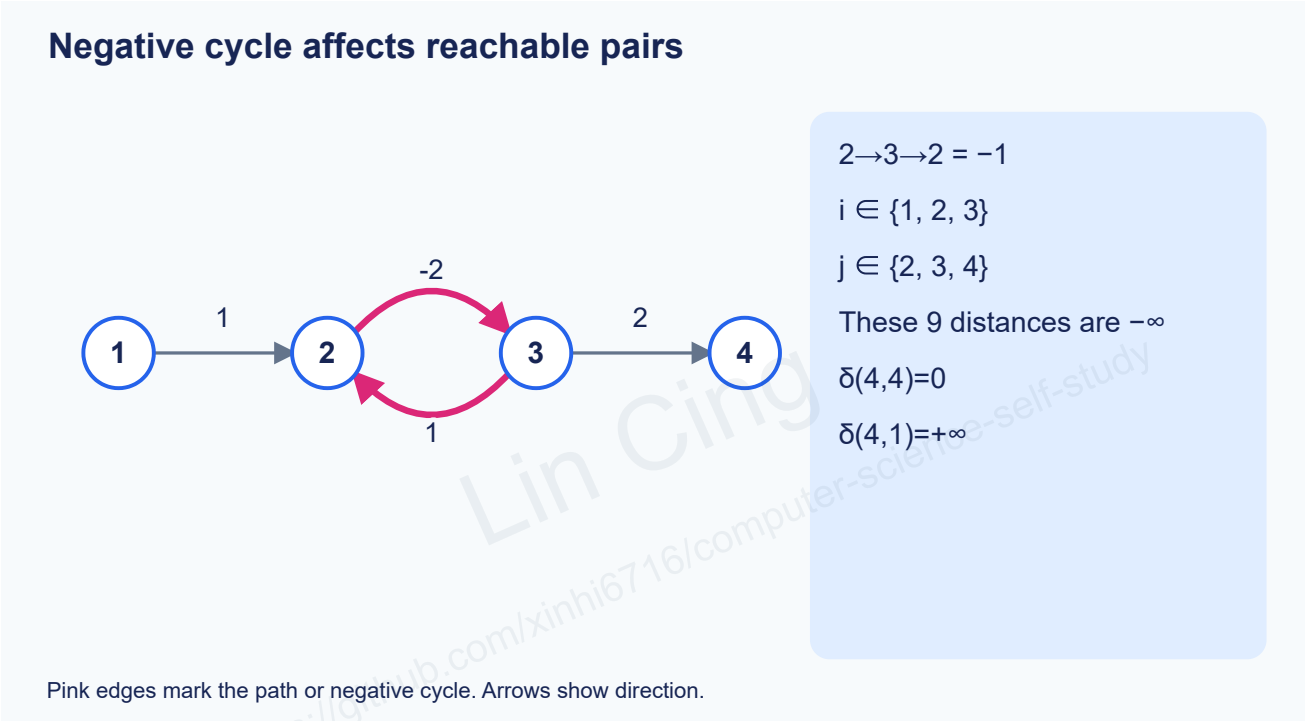
After Floyd, some D[t,t]<0 if and only if a negative cycle exists. A negative diagonal gives a negative closed walk, which contains a negative cycle. Conversely, allowing all intermediates exposes a negative diagonal at some vertex on a negative cycle.

Use a reachability matrix R to identify affected pairs.

$$\delta(i,j) = -\infty \iff \exists t:\ R[i,t] \land D[t,t] < 0 \land R[t,j].$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A negative diagonal warns; mark −∞ by pair (continued)




This separate counterexample is not the shared graph G. Exactly {1,2,3}×{2,3,4}, nine pairs, are affected. Complete finite arithmetic before marking. With negative cycles, finite Floyd values are not unrestricted distances. Johnson instead reports a negative cycle and stops globally. [S1 ex25.2-6, § 25.3]

# Johnson obtains potentials from a super-source

Add s and a zero-weight edge $s \to v$ to every vertex v. No edge enters s, so it creates no cycle and cannot change paths between original vertices. All original vertices are reachable from s, so Bellman–Ford detects any negative cycle.

Only if no negative cycle exists, set $h(v)=\delta(s,v)$. On the shared graph $h=(0,0,-2,0)$: $s \to 2 \to 3$ gives $-2$; no other vertex drops below 0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Bellman–Ford stages establish the potentials

Synchronous Bellman–Ford stages use order (s,1,2,3,4).

| Edge budget | Distance vector |
|---|---|
| 0 | $(0,\infty,\infty,\infty,\infty)$ |
| 1 | $(0,0,0,0,0)$ |
| 2 | $(0,0,0,-2,0)$ |
| 3 and 4 | $(0,0,0,-2,0)$ |

After $|V'|-1=4$ stages, stage 5 has no improvement. With in-place edge relaxations, intermediate vectors depend on edge order, while final potentials agree. [S1 p702–704; S2 p22–23; S3 p22–23]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The shortest-path triangle inequality makes weights nonnegative

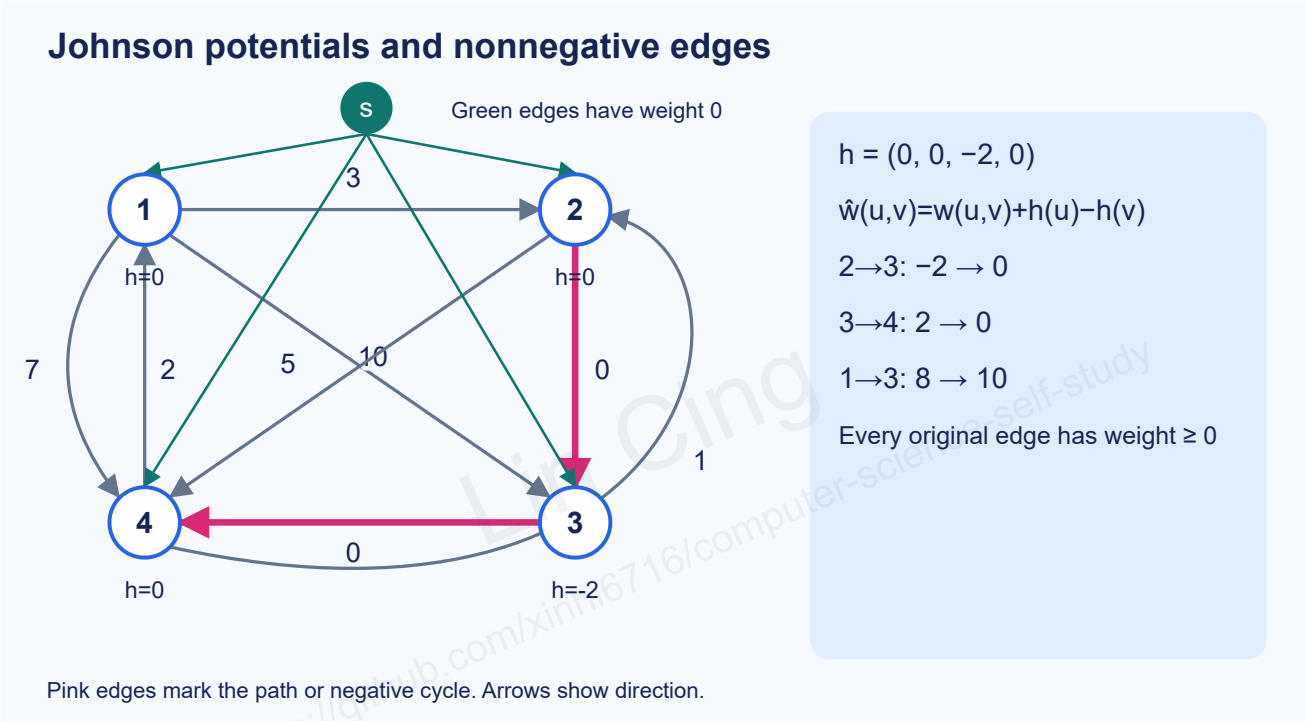
Bellman–Ford potentials satisfy $h(v) \le h(u) + w(u,v)$. Rearranging gives

$$\widehat{w}(u,v) = w(u,v) + h(u) - h(v) \ge 0.$$

| u→v | w | h(u)−h(v) | ŵ |
|---|---|---|---|
| 1→2 | 3 | 0 | 3 |
| 1→3 | 8 | 2 | 10 |
| 1→4 | 7 | 0 | 7 |
| 2→3 | -2 | 2 | 0 |
| 2→4 | 5 | 0 | 5 |
| 3→4 | 2 | -2 | 0 |
| 4→1 | 2 | 0 | 2 |
| 4→2 | 1 | 0 | 1 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The shortest-path triangle inequality makes weights nonnegative (continued)




Any h preserves path order. Only h satisfying the difference constraints guarantees nonnegative edges. Adding the same constant to every edge is not a substitute, because paths can have different edge counts. [S1 lemma25.1, p702; S2 p19–23; S3 p19–23]

# Telescoping preserves the order of paths with fixed endpoints

For $p=\langle v_0,\ldots,v_r\rangle$,

$$\widehat{w}(p)=\sum_{a=1}^{r}(w(v_{a-1},v_a)+h(v_{a-1})-h(v_a))=w(p)+h(v_0)-h(v_r).$$

Internal potentials cancel. Every path with the same endpoints receives the same constant, preserving strict order, ties, and the complete set of shortest paths. A cycle has equal endpoints, so its weight is unchanged. A negative cycle cannot become nonnegative.

For example, direct 1→3 has weight 8 and reweighted weight 10. Path 1→2→3 has weight 1 and reweighted weight 3. Both increase by 2. For 1→2→3→4 and direct 1→4, the endpoint potential difference is 0; weights remain 3 and 7.

Thus $\delta(u,v)=\widehat{\delta}(u,v)-h(u)+h(v)$. Check reachability first. Keep ∞ unchanged rather than applying signed integer arithmetic to a sentinel. [S1 lemma25.1; S2 p19–21; S3 p19–21]

# Run Dijkstra from every source, then restore distances

From source 1, vertices settle in order 1,2,3,4. After extracting 1, estimates are (0,3,10,7). After 2, they are (0,3,3,7). After 3, they are (0,3,3,3).

All sources produce these reweighted distances.

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 3 | 3 |
| 2 | 2 | 0 | 0 | 0 |
| 3 | 2 | 1 | 0 | 0 |
| 4 | 2 | 1 | 1 | 0 |

Apply $\delta = \tilde{\delta} - h(\text{source}) + h(\text{destination})$ to each cell to recover $D^{(4)}$. For example, $\delta(1,3) = 3 - 0 - 2 = 1$ and $\delta(3,1) = 2 - (-2) + 0 = 4$. Keep the Dijkstra predecessor paths because reweighting preserves paths.

The checker also enumerates simple paths to verify all 16 entries. Agreement between two DPs is not its only evidence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Density and heap choice determine cost

| Method | Time | Conditions |
| --- | --- | --- |
| Slow min-plus | $\Theta(n^4)$ | Edge-budget DP for derivation |
| Repeated squaring | $\Theta(n^3 \log n)$ | Min-plus associativity |
| Floyd–Warshall | $\Theta(n^3)$ | Matrix input and dense graphs |
| Johnson, binary heap | $O(n(m+n)\log n)$ | Sparse adjacency lists |
| Johnson, Fibonacci heap | $O(nm+n^2\log n)$ | Amortized $O(1)$ decrease-key |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Compare total cost and working space

With an explicit super-source, Bellman–Ford costs $O(n(m+n))$. Add $n$ Dijkstra runs and $\Theta(n^2)$ restoration. With Fibonacci heaps, $m=\Theta(n^2)$ gives $O(n^3)$; $m=O(n)$ gives $O(n^2 \log n)$. The binary-heap dense upper bound is $O(n^3 \log n)$. A full output already needs $\Theta(n^2)$ space.

Floyd uses $\Theta(n^2)$ working space; retaining every teaching stage uses $\Theta(n^3)$. Johnson uses $O(m+n)$ for adjacency lists and one source beyond the output.

Source keys: S0=original notes p18–19; S1=CLRS 3e § 25.1–25.3, printed p684–705; S2=MIT 2004 lecture16 p1–23; S3=MIT 2005 lecture19 p1–23. See sources.json for paths, read pages, and content mappings.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapters 23–25. MIT 6.046J, Fall 2004, Lectures 13–16; Fall 2005, Lectures 16–19. Original handwritten algorithms notes (演算法_wjungle). These notes provide selected proof ideas and context; complete worked traces and diagrams are original educational synthesis.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing https://github.com/xinhi6716/computer-science-self-study